WAP in python/C++ language to turn ON/OFF buzzer.

```
// Define the buzzer pin
const int buzzerPin = 9;  // Connect the piezo buzzer to pin 9

// Define the frequency for the "tick" sound (in Hz)
const int tickFrequency = 1000;  // Frequency of the tick sound (1000 Hz)

// Define the duration for the "tick" sound
const int tickDuration =200;  // Duration of each tick sound (in milliseconds)

void setup() {
  // Set the buzzer pin as an output
  pinMode(buzzerPin, OUTPUT);
}
```

OR

```
const int buzzerPin = 9; // Connect the positive pin of the buzzer to pin
9

void setup() {
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  tone(buzzerPin, 1000); // Play a 1 kHz tone
  delay(100);            // Duration of the tick
  noTone(buzzerPin);     // Stop the tone
  delay(900);            // Wait before the next tick
}
```

WAP in python/C++ language to blink LED.

```
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
```

```
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

WAP in python/C++ language to toggle two LED's.

```
/ Define the LED pins
const int led1 = 13;  // Built-in LED (usually on pin 13)
const int led2 = 2;   // External LED connected to pin 2

void setup() {
  // Initialize both pins as output
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
}

void loop() {
  // Turn on LED1 and turn off LED2
  digitalWrite(led1, HIGH);
  digitalWrite(led2, LOW);
  delay(1000); // Wait for 1 second

  // Turn off LED1 and turn on LED2
  digitalWrite(led1, LOW);
  digitalWrite(led2, HIGH);
  delay(1000); // Wait for 1 second
}
```

Observations:

1. Input: Any obstacle that comes in the range of the sensor is detected. IR sensors actually measure the heat being emitted from the object. So the heat is the actual input for the sensors. The IR sensor gets its input from GPIO 21.
2. Output: The output is shown by the LED's and the buzzer. When an obstacle is detected, the Green LED glows

Conclusion: We have successfully implemented the connection of IR sensor with Arduino for obstacle detection. The output is shown by a glowing buzzer.

Pin mode

Description

Configures the specified pin to behave either as an input or an output.

See the Digital Pins page for details on the functionality of the pins.

It is possible to enable the internal pullup resistors with the mode

`INPUT_PULLUP`. Additionally, the `INPUT` mode explicitly disables the internal pullups.

Syntax

```
pinMode(pin, mode)
```

Parameters

> `pin`: the Arduino pin number to set the mode of.
> `mode`: `INPUT`, `OUTPUT`, or `INPUT_PULLUP`. See the Digital Pins page for a more complete description of the functionality.

Returns

Nothing

digitalRead()

Description

Reads the value from a specified digital pin, either

`HIGH` or `LOW`.

Syntax

`digitalRead(pin)`

Parameters

`pin`: the Arduino pin number you want to read

Returns

`HIGH` or `LOW`

Description

Write a

`HIGH` or a `LOW` value to a digital pin.
If the pin has been configured as an

`OUTPUT` with `pinMode()`, its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for `HIGH`, 0V (ground) for `LOW`.
If the pin is configured as an

digitalwrite()

`INPUT`, `digitalWrite()` will enable (`HIGH`) or disable (`LOW`) the internal pullup on the input pin. It is recommended to set the `pinMode()` to `INPUT_PULLUP` to enable the internal pull-up resistor. See the Digital Pins tutorial for more information.

If you do not set the

`pinMode()` to `OUTPUT`, and connect an LED to a pin, when calling `digitalWrite(HIGH)`, the LED may appear dim. Without explicitly setting `pinMode()`, `digitalWrite()` will have enabled the internal pull-up resistor, which acts like a large current-limiting resistor.

Syntax

```
digitalWrite(pin, value)
```

Parameters

`pin`: the Arduino pin number
`value`: `HIGH` or `LOW`

Returns

Nothing

delay()

Description

Pauses the program for the amount of time (in milliseconds) specified as parameter. (There are 1000 milliseconds in a second.)

Syntax

```
delay(ms)
```

Parameters

`ms`: the number of milliseconds to pause. Allowed data types: `unsigned long`.

Returns

Nothing

LED_BUILTIN

Defining built-ins: LED_BUILTIN

Most Arduino boards have a pin connected to an on-board LED in series with a resistor. The constant

`LED_BUILTIN` is the number of the pin to which the on-board LED is connected. Most boards have this LED connected to digital pin 13.

Defining Pin Levels: HIGH and LOW

When reading or writing to a digital pin there are only two possible values a pin can take/be-set-to:

`HIGH` and `LOW`. These are the same as `true` and `false`, as well as `1` and `0`.

HIGH

The meaning of

`HIGH` (in reference to a pin) is somewhat different depending on whether a pin is set to an `INPUT` or `OUTPUT`. When a pin is configured as an `INPUT` with `pinMode()`, and read with `digitalRead()`, the Arduino board (ATmega) will report `HIGH` if:

> a voltage greater than 3.0V is present at the pin (5V boards)
> a voltage greater than 2.0V is present at the pin (3.3V boards)

A pin may also be configured as an INPUT with

`pinMode()`, and subsequently made HIGH with `digitalWrite()`. This will enable the internal 20K pullup resistors, which will *pull up* the input pin to a `HIGH` reading unless it is pulled `LOW` by external circuitry. This can be done alternatively by passing `INPUT_PULLUP` as argument to the `pinMode()` function, as explained in more detail in the section "Defining Digital Pins modes: INPUT, INPUT_PULLUP, and OUTPUT" further below.

When a pin is configured to OUTPUT with

`pinMode()`, and set to `HIGH` with `digitalWrite()`, the pin is at:

> 5 volts (5V boards)
> 3.3 volts (3.3V boards)

In this state it can source current, e.g. light an LED that is connected through a series resistor to ground.

LOW

The meaning of LOW also has a different meaning depending on whether a pin is set to INPUT or OUTPUT. When a pin is configured as an INPUT with pinMode(), and read with digitalRead(), the Arduino board (ATmega) will report LOW if:

a voltage less than 1.5V is present at the pin (5V boards)
a voltage less than 1.0V (Approx) is present at the pin (3.3V boards)

When a pin is configured to OUTPUT with pinMode(), and set to LOW with digitalWrite(), the pin is at 0 volts (both 5V and 3.3V boards). In this state it can sink current, e.g. light an LED that is connected through a series resistor to +5 volts (or +3.3 volts).